DA2304 Assignment 3 – Report

Athreya Ganesh

DA24B002

Extra Credits

While building the parser, I did not come across much errors debugging wise, but I built the compilation engine before building the vm writer, and had to make the changes all over again. When translating VM files to ASM and trying them out, I realised I had made some syntactical errors (A=M+D, etc.) and corrected them in my assignment 2's VM to ASM translator before running it again.

Part 1: The Jack Convolution Program

Exercise 1: 2D Convolution – Background

I have configured `stride = 1` in my program `Conv.jack`, and have chosen a standard edge-detection kernel to convolve on the input matrix. The kernel chosen is:

$$F = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Exercise 2: The Conv class in Jack

Class Skeleton

Using the given skeleton, in Jack programming language, I programmed the Conv class. The code is attached below:

```
class Conv {
    static Array ArrayFilter; // 3x3 filter , flattened to length 9
    static int stride; // initialised in constructor (1 or 2)

    field Array ArrayInput; // input matrix , flattened
    field Array ArrayOutput; // output matrix , flattened
    field int ArraySize; // side length N of input (NxN)
```

```
field int OutputSize; // dimension of output
// Filter is always of size 3.

/** Constructor : initialise stride and allocate output . */
constructor Conv new (int N) {
    let ArraySize = N;
    let stride = 1;
    let OutputSize = calcOutputDim();
    let ArrayInput = Array.new(N*N);
    if (OutputSize > 0){
        let ArrayOutput = Array.new(OutputSize * OutputSize);
    }

    let ArrayFilter = Array.new(9);
    return this;
}

/** Initialise the 3x3 filter with given values . */
method void initFilter (Array values) {
    var int i;
    let i = 0;
    while (i<9){
        let ArrayFilter[i] = values[i];
        let i = i + 1;
    }
    return;
}

/** Read input matrix entries from the caller . */
method void getArrayInput (Array input, int size) {
    var int i;
    var int totalSize;

    let ArraySize = size;
    let totalSize = size * size;
    let i = 0;

    while (i < totalSize){
        let ArrayInput[i] = input[i];
        let i = i + 1;
    }
    return;
}

/** Compute output dimension using the conv formula . */
method int calcOutputDim () {
    var int O;
    if (ArraySize < 3){
```

```
        return 0;
    }
    let O = ((ArraySize - 3)/stride) + 1; //padding = 0
    return O;
}

/** Run the full convolution and store result in ArrayOutput . */
method void conv2d () {
    var int r, c, i, j;
    var int outIdx, inIdx, fIdx;
    var int sum;
    if (ArraySize < 3){
        do Memory.poke(8000, 1);
        return;
    }
    let r = 0;
    let outIdx = 0;

    while (r < OutputSize){
        let c = 0;
        while (c < OutputSize){
            let sum = 0;
            let i = 0;

            //MAC IMPLEMENTATION
            while (i < 3){
                let j = 0;
                while (j < 3){
                    let inIdx = (((r * stride) + i) * ArraySize) + ((c * stride) + j);
                    let fIdx = (i * 3) + j;
                    let sum = sum + (ArrayInput[inIdx] * ArrayFilter[fIdx]);
                    let j = j + 1;
                }
                let i = i + 1;
            }
            let ArrayOutput[outIdx] = sum;
            let outIdx = outIdx + 1;
            let c = c + 1;
        }
        let r = r + 1;
    }
    return;
}

/** Print the output matrix to screen . */
method void printOutput () {
    var int r, c, idx;
    let r = 0;
```

```
        let idx = 0;
        while (r < OutputSize){
            let c = 0;
            while (c < OutputSize){
                do Output.printInt(ArrayOutput[idx]);
                do Output.printChar(32);
                let idx = idx + 1;
                let c = c + 1;
            }
            do Output.println();
            let r = r + 1;
        }
        return;
    }

    /** Free allocated memory . */
    method void dispose () {
        if (~(ArrayInput = null)) {
            do ArrayInput.dispose();
        }
        if (~(ArrayOutput = null)) {
            do ArrayOutput.dispose();
        }
        // Free the object itself
        do Memory.deAlloc(this);
        return;
    }
}
```

Functional Requirements

- **Constructor:** In the constructor, we initialize the `ArraySize` to N, `stride` to 1. Given that filter is always of size 3, we directly calculate the size of the output matrix using the formula given, and assign `ArrayOutput` to a new empty matrix, and `ArrayFilter` to a new matrix. Note that all arrays have been flattened to a single dimension.
- `initFilter`: This method takes values from the argument and assigns them to the filter matrix `ArrayFilter`. I used a common edge detection kernel.
- `getArrayInput`: This takes an input matrix from the argument, along with its size, and assigns these values to the input matrix `ArrayInput`.
- `calcOutputDim`: This implements the formula mentioned: $O = \lfloor \frac{N-3+2 \cdot P}{S} \rfloor + 1$ using Jack arithmetic.
- `convolve`: This is implemented as the function conv2d() which takes no arguments, and runs the entire convolution with MAC implementation, and stores the result in `ArrayOutput`.
- `printOutput`: This prints the result of the convolution `ArrayOutput` like a matrix.

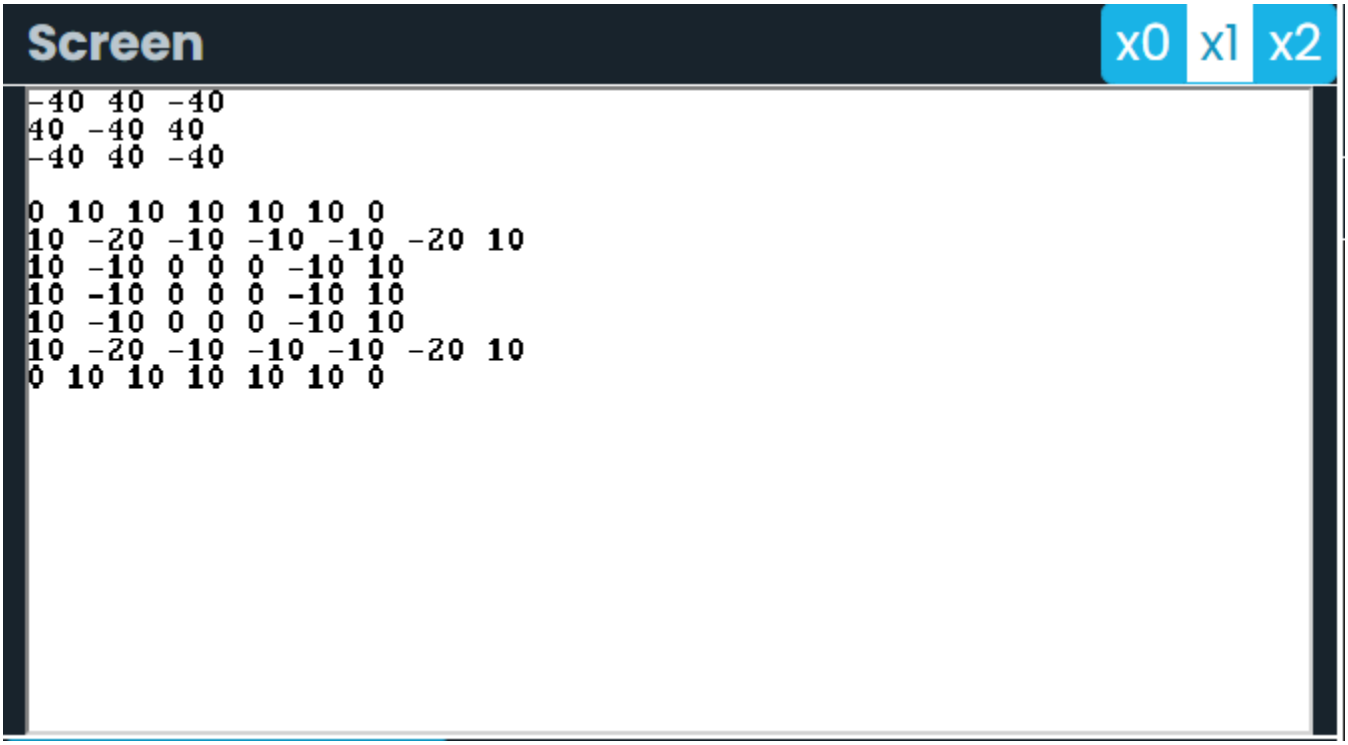
Driver Program

The implemented Main.jack is given below:

```
class Main{
    function void main(){
        var Array inputMatrix;
        var Array filterMatrix;
        var Conv convObj;
        var int i;
        var int row, col;
        var Array inputMatrix9x9;

        let inputMatrix = Array.new(25);
        let i = 0;

        while (i < 25) {
            // Alternates values between 0 and 10 based on even/odd index
            if ((i - ((i / 2) * 2)) = 0) {
                let inputMatrix[i] = 10;
            } else {
                let inputMatrix[i] = 0;
            }
            let i = i + 1;
        }

        let filterMatrix = Array.new(9);

        //Filter is:
        //0  1  0
        //1 -4  1
        //0  1  0

        let filterMatrix[0] = 0;
        let filterMatrix[1] = 1;
        let filterMatrix[2] = 0;
        let filterMatrix[3] = 1;
        let filterMatrix[4] = -4;
        let filterMatrix[5] = 1;
        let filterMatrix[6] = 0;
        let filterMatrix[7] = 1;
        let filterMatrix[8] = 0;

        let convObj = Conv.new(5);
        do convObj.initFilter(filterMatrix);
        do convObj.getArrayInput(inputMatrix, 5);
        do convObj.conv2d();
        do convObj.printOutput();
        do Output.println();

        do convObj.dispose();
        do inputMatrix.dispose();
```

```
        //EXTRA CREDIT//

        let inputMatrix9x9 = Array.new(81);
        let i = 0;
        while (i < 81){
            let row = i / 9;
            let col = i - (row * 9);

            if ((row > 1) & (row < 7) & (col > 1) & (col < 7)) {
                let inputMatrix9x9[i] = 10;
            }
            else {
                let inputMatrix9x9[i] = 0;
            }
            let i = i + 1;
        }

        let convObj = Conv.new(9);
        do convObj.initFilter(filterMatrix);
        do convObj.getArrayInput(inputMatrix9x9, 9);
        do convObj.conv2d();
        do convObj.printOutput();

        do convObj.dispose();
        do inputMatrix9x9.dispose();
        do filterMatrix.dispose();

        return;
    }
}
```

This driver program creates a 5×5 matrix, convolves it, displays the output, and disposes of it neatly. The same is done for a 9×9 matrix as well. The results displayed on the screen are as follows:



This correctly matches with the expected result, and hence we conclude that our program runs successfully.

Part 2: The Jack Compiler Frontend

Exercise 1: The Tokenizer (Lexical Analyzer)

The provided `JackTokenizer.py` converts a given jack program to a stream of tokens. I have uploaded JackTokenizer.py along with other deliverables in the folder src.

Exercise 2: The Parser (Syntax Analyzer + VM Code Generator

I have implemented `SymbolTable.py`, `VMWriter.py`, `CompilationEngine.py`, which work together to compile the given jack program and convert it to VM code, all of which are unified in `JackCompiler.py` which implements the compiler I made. I have placed the output files in the `out` folder, along with the `.asm` translation from Assignment 2's translator.

Exercise 3: Analysis Questions

1. **Tokenizer Design:** Regex fails when there is a line of code in between two comments, or when there are multi-line comments. For example. `/*comment*/ let x = 5; /*comment*/` here, the statement `let x = 5;` would get eaten up by the regex. When the comment spans

multiple lines, standard regex stops when a line ends and hence fails. Therefore, a state based tracking approach is ideal here.

2. **Symbol Table:**

Identifier	Scope	Type	Kind
ArrayFilter	Class	Array	static
stride	Class	int	static
ArrayInput	Class	Array	field
ArrayOutput	Class	Array	field
ArraySize	Class	int	field
OutputSize	Class	int	field
this	Subroutine	Conv	arg
r	Subroutine	int	var
c	Subroutine	int	var
i	Subroutine	int	var
j	Subroutine	int	var
outIdx	Subroutine	int	var
inIdx	Subroutine	int	var
fIdx	Subroutine	int	var
sum	Subroutine	int	var

3. **Compilation Correctness:**

```
push this 1      // Pushes base memory address of ArrayOutput (field index 1)
push local 4     // Pushes value of outIdx (local index 4)
add              // Adds them
push local 7     // Pushes value of sum (local index 7)
pop temp 0       // Pop expression value safely into temp 0
pop pointer 1    // Pop the destination RAM address into pointer 1 (sets THAT segment)
push temp 0      // Put the expression value back to top of stack
pop that 0       // Store value into THAT[0]
```

Here we put the object's `ArrayOutput` base address on the stack after the offset `outIdx`, add them to find the target RAM address. We store the required sum in temp 0, then pop destination RAM address to pointer 1, it makes THAT point to the required memory segment. Then, we push the sum and store it in THAT[0].

4. **Convolution Stride:** For a 9×9 (N=9) input with F=3 and S=2, the output dimension is:

$$O = \lfloor \frac{N - F + 2P}{S} \rfloor + 1 = \lfloor \frac{9 - 3 + 2(0)}{2} \rfloor + 1$$
$$= 3 + 1 = 4$$

Modifying the stride from S=1 to S=2 does not change the asymptotic VM instruction count of convolve(), since the number of instructions is approximately $\mathcal{O}(N^2F^2) = \mathcal{O}(N^2)$ due to the

four while loops involved. Changing from S=1 to S=2 reduces the number of steps involved roughly by a factor of 4, but does not affect the quadratic growth rate with respect to N.